

03 Conditions, cycles and libraries in Python

Introduction to Python – RSE course

What did we learn last week?

- **variables** store values
- every variable has a **data type** (string, int...)
- **data structures** store more values (list, tuple, dict...)
- **functions** group instructions into reusable blocks
- functions can have **parameters** (data in) and **return** (data out)
- **local variables** live only inside a specific function
- Python provides **built-in functions** that help us write simpler and shorter code

Conditions

- writing more **complex code**
- allow us to make decisions in our code based on certain criteria
- **if statement** is a *conditional* statement that controls whether a block of code is executed or not
- **else statement** can be used following **if** to allow us to specify an alternative code block to execute when the *if branch* is not taken
- **elif** (short for “else if”) to provide several alternative choices, each with its own test

Often, you want some combination of things to be true - You can combine relations within a **conditional using **and** and **or****

Loops

- A *for loop* tells Python to execute some statements once for each value in a list, a character string, or some other collection
- “**for each thing in this group, do these operations**”
- The built-in function `range()` produces a sequence of numbers.
 - *Not* a list: the numbers are produced on demand to make looping over large ranges more efficient.
 - `range(N)` is the numbers 0..N-1
- A *while loop* is used when you want to **repeat while** a condition is true

Indentation is always meaningful in Python!

```
firstName = "Jon"  
  lastName = "Smith"
```

```
File "<ipython-input-7-f65f2962bf9c>", line 2  
    lastName = "Smith"  
    ^  
IndentationError: unexpected indent
```

Working with files

- **reading** and **writing** to files
- open the file with function `open()`
 - **r** - read
 - **w** - write (overwrite)
 - **a** - append
 - recommended using **with** - the file is automatically closed

Libraries – NumPy, math

- well-tested algorithms and tools
- **NumPy** - library for numerical computations and working with arrays
 - numpy **arrays** - more efficient, support vectorized operations (on entire array at once)
 - functions like **mean**, **sum** or **standard deviation**
 - we can handle **NaN values** (missing values)

NumPy is foundation for **data analysis in Python** - follow-up course next week

Key takeaways

- **conditions** allow us make **decisions** in code
 - combine conditions with **and** or **or**
 - **for** and **while** loops for repeating code
 - working with files - **with open(...)**
 - NumPy library for numerical computations
-